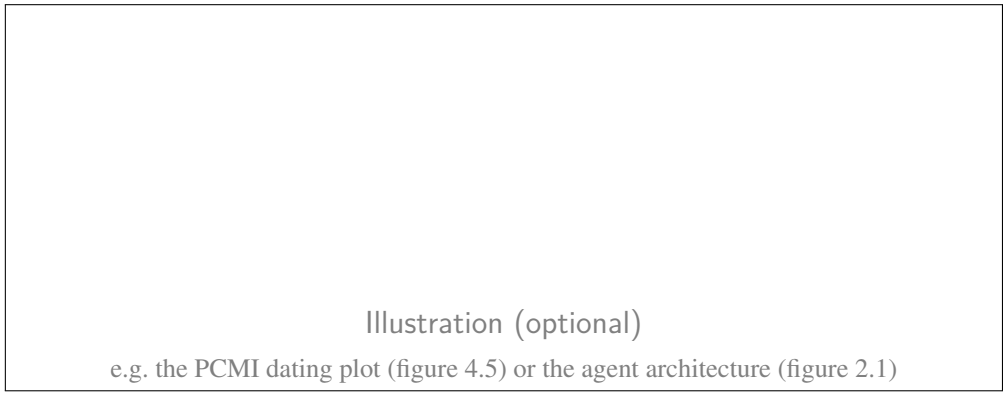
Research Project (PRe)

Specialty: STIC

Academic year: 2025–2026

A Conversational Agent for Nuclear Fuel Performance Simulation

Automating the OFFBEAT code and building a digital twin of a PWR fuel rod



Confidentiality statement

Non-confidential report

(state the type of confidentiality and, where applicable, its end date)

Author: Yann BUTEL **Class of: 2027**

ENSTA Paris supervisor:
PARICAUD Patrice

Host organisation supervisor:
GONG Helin

Internship carried out from 13 April 2026 to 1 July 2026

Host organisation: SPEIT
Address: Minhang Campus, No.800, Dongchuan Road

Non-confidentiality notice

This document is **non-confidential**. It may therefore be consulted online by anyone within the ENSTA Paris community. It will be archived electronically by the ENSTA Paris library.

Non-confidential work: “the document is non-confidential. It may therefore be consulted online by anyone.”

The (non-)confidentiality statement signed by the host organisation supervisor is attached to this report.

Acknowledgements

I would like to thank Mr GONG Helin for welcoming me at SPEIT, for proposing an open and multidisciplinary subject, and for being responsive to my needs.
I also thank Mr PARICAUD Patrice, who gave me his time to conduct this internship.

Abstract

Nuclear fuel performance simulation relies on codes whose use remains restricted to specialists. This research project consisted in designing **AutoOFFBEAT**, a conversational agent based on a large language model, which automates the simulation workflows of the **OFFBEAT** code (built on OpenFOAM): a supervisor orchestrates domain tools (case generation, execution, post-processing, safety analysis) and a self-healing loop diagnoses solver crashes from an editable knowledge base. The system was then extended towards a **digital twin** of a fuel rod, including a Gaussian-process surrogate that predicts pellet-cladding gap closure within 4.4 %, in 2 ms instead of 36 s, for a linear power absent from its training set. The second part of the work is the **measured stress-testing** of the system: a benchmark of twelve injected faults shows that self-healing only handles crashes whose root cause the available correction actually addresses, and evaluating the decision layer raises tool-selection accuracy from 77 % to 88 %. These measurements exposed fourteen defects, none of which raised any visible error (including a confusion between cladding and pellet temperature (830 K discrepancy) and a surrogate validated over a degenerate domain).

Keywords:
Nuclear fuel performance ; OFFBEAT ; OpenFOAM ; LLM agent ; digital twin ; safety analysis ; pellet-cladding mechanical interaction (PCMI) ; surrogate model ; Gaussian process.

Résumé

La simulation de performance du combustible nucléaire repose sur des codes dont la mise en œuvre reste réservée à des spécialistes. Ce projet de recherche a consisté à concevoir **AutoOFFBEAT**, un agent conversationnel fondé sur un grand modèle de langage, qui automatise les chaînes de calcul du code **OFFBEAT** (bâti sur OpenFOAM) : un superviseur orchestre des outils de domaine — génération de cas, exécution, post-traitement, analyse de sûreté — et une boucle d'auto-réparation diagnostique les plantages du solveur à partir d'une base de connaissance éditable. Le système a ensuite été étendu vers un **jumeau numérique** de crayon combustible, incluant un émulateur par processus gaussien qui prédit la date de fermeture du jeu pastille-gaine à 4.4 % près, en 2 ms au lieu de 36 s, pour une puissance absente de son apprentissage. Le second volet du travail est la **mise à l'épreuve mesurée** du système : un banc de douze fautes injectées montre que l'auto-réparation ne traite que les plantages dont la cause racine relève du correctif disponible, et l'évaluation de la couche de décision porte l'exactitude de sélection des outils de 77 % à 88 %. Ces mesures ont révélé quatorze défauts dont aucun ne provoquait d'erreur visible — dont une confusion entre température de gaine et de pastille (830 K d'écart) et un émulateur validé sur un domaine dégénéré.

Mots-clés :
Performance combustible nucléaire ; OFFBEAT ; OpenFOAM ; agent LLM ; jumeau numérique ; analyse de sûreté ; interaction pastille-gaine (PCMI) ; modèle réduit ; processus gaussien.

Contents

List of Figures

List of Tables

List of Appendices

Introduction

Context of the internship

In a nuclear power plant, the management of the fuel rod governs the safety of the installation. A fuel rod is a tube a few metres long, made of a stack of uranium oxide pellets enclosed in a metallic cladding. It must operate for several years under extreme conditions — temperatures above 1000 K, intense irradiation, fuel swelling, cladding creep, and more. Predicting its evolution is therefore essential in order to minimise risk.

OFFBEAT (*OpenFOAM Fuel BEhavior Analysis Tool*) is one of the computational codes dedicated to this field. Developed chiefly at the Laboratory for Reactor Physics and Systems Behaviour of EPFL, at the Paul Scherrer Institute and at Texas A&M University, it is built on the open-source OpenFOAM library and solves, using the finite volume method, the coupled thermal and mechanical behaviour of a rod in one, two or three dimensions [1]. Its use follows the logic of OpenFOAM: a case is a tree of dictionary files that the user must know how to write, a mesh to be generated, a solver to be launched from the command line, and VTK-format outputs to be post-processed. The main difficulty with OFFBEAT is that this chain of tasks, while powerful, is not very accessible.

Subject and objective

The objective set at the start of the internship was to *"build an agent on the nuclear fuel performance code OFFBEAT"*. The task was therefore to design a system able to bridge a request expressed in natural language and the actual execution of an OFFBEAT simulation, taking inspiration from the open-source project **AutoFLUKA** [3], which applies this idea to the particle transport code FLUKA.

The project was carried out in two stages. The first consisted in building the **base agent**: a supervisor built on a large language model, orchestrating domain tools able to create a case, launch the solver, automatically repair certain crashes and post-process the results. The second stage extended the system towards a **digital twin** of the fuel rod. This meant monitoring safety criteria as new data arrive, predicting threshold crossings, and returning instantaneous predictions through a reduced-order model.

Outline of the report

The report follows the progression of the work. **Chapter 1** presents the scientific and technical context: the physics of the fuel rod, the OFFBEAT code, and the principles of agents built on language models. **Chapter 2** describes the architecture of the base agent and details its components, in particular the self-healing loop, which is its most distinctive contribution. **Chapter 3** sets out the extension towards the digital twin through the five building blocks developed. **Chapter 4** gathers the results: a reference simulation over a complete irradiation cycle, the validation of the surrogate, and above all the systematic stress-testing of the agent's self-diagnosis capabilities. **Chapter 5** returns to the difficulties encountered and the false starts, whose analysis drove several important corrections, and presents the internship schedule. The conclusion examines the possible directions

for continuing the work.

Chapter 1

Scientific and technical context

1.1 The physics of the fuel rod

1.1.1 Anatomy and loadings

A fuel rod in a pressurised water reactor (PWR) is a Zircaloy tube, called the *cladding*, roughly 9.5 mm in outer diameter and a few metres long, inside which a column of cylindrical uranium dioxide (UO_2) pellets is stacked. Between pellet and cladding there initially remains a radial gap of a few tens of micrometres, filled with pressurised helium.

This gap is central to the behaviour of the rod. It constitutes a thermal resistance that the heat must cross before reaching the cladding. A radial temperature gradient of several hundred kelvins therefore appears between the centre of the pellet and its periphery.

Under irradiation, several coupled phenomena modify this geometry:

- the **densification** and then the **swelling** of the fuel, under the effect of solid and gaseous fission products;
- the **relocation** of the fragments of the cracked pellet;
- the **creep** of the cladding, which slowly deforms under the coolant pressure (about 15.5 MPa in a PWR);
- the **release of fission gases** (xenon, krypton), which degrades the conductivity of the gap and raises the internal pressure.

The combined effect is a **progressive closure of the gap**. Once contact is established, the pellet mechanically loads the cladding: this is the **pellet-cladding mechanical interaction** (PCMI).

1.1.2 Safety criteria

The design of a fuel rod rests on quantified safety criteria. Table 1.1 summarises those relevant here, explicitly separating the ones a fuel performance code computes from those belonging to other disciplines. This distinction proved essential when scoping the digital twin (§3.1).

Note that the DNBR belongs to core thermal-hydraulics, outside the domain of OFFBEAT. For the code it is a boundary condition.

1.2 The OFFBEAT code

1.2.1 Position among fuel performance codes

Two distinct generations of codes coexist in fuel performance simulation.

Table 1.1: Fuel rod safety criteria and their coverage by OFFBEAT. The values given are design orders of magnitude, to be confirmed according to the rod and the burnup considered.

Criterion	Indicative limit	Quantity	OFFBEAT
Pellet centreline melting	$T < 3113\,\mathrm{K}$ (fresh UO_2; decreases with burnup)	Maximum temperature	yes
Cladding temperature (PCT, LOCA criterion)	$< 1477\,\mathrm{K}$ (1204 °C)	Temperature in the cladding	if transient modelled
Cladding strain (PCMI)	circumferential strain $< 1\,\%$	$\theta\theta$ strain	yes
Gap closure	margin > 0	Gap width	yes
Internal pressure (*no-liftoff*)	$< P_{\mathrm{coolant}}$ ($\approx 15.5\,\mathrm{MPa}$)	Plenum pressure	if FGR model enabled
Cladding oxidation (ECR)	$< 17\,\%$	Oxide thickness	model dependent
Departure from nucleate boiling (DNBR)	> 1.3	—	**no** (input)

The first gathers the so-called **1.5D** codes, which discretise the rod into independent axial slices coupled only through the plenum and the internal pressure. **FRAPCON** and its transient counterpart **FRAPTRAN**, and **TRANSURANUS**, the European solution, belong to this category [4, 5]. Their strength is speed and robustness: they are validated against very large experimental databases and remain the reference tools for design studies. Their limitation is geometric: azimuthal cracking, a pellet edge or a non-axisymmetric gradient escape them by construction.

The second generation is **multidimensional**. **BISON**, **ALCYONE**, the French solution, and **OFFBEAT** belong to it. These codes solve the rod in two or three dimensions and therefore capture local effects, at the cost of a far higher computational burden (of the order of three orders of magnitude compared with a 1.5D code).

OFFBEAT occupies a particular position in this landscape, and this is what justifies its choice here: it is **multidimensional**, **open source**, and built on OpenFOAM, a widely used library. Code-to-code comparisons moreover show good agreement between OFFBEAT and BISON [2].

1.2.2 Solution principle

On an unstructured mesh, OFFBEAT solves the coupling between:

- a **thermal problem**: conduction in the pellet and the cladding, transfer across the gap by gas conduction and radiation;
- a **mechanical problem**: elasticity, plasticity and creep, in a cell-centred finite volume formulation;
- **behavioural models**: burnup, swelling, densification, fission gas release, pellet-cladding contact.

An OFFBEAT case inherits the structure of an OpenFOAM case: a `0/` directory for the initial fields, `constant/` for the properties and models, `system/` for the run control. To these is added a `rodDict` file, specific to OFFBEAT, which describes the rod geometry block by block and from which a script generates the mesh. The detail of this tree is given in appendix Appendix B. Nothing in this chain is tolerant to error, and whatever the source of a crash, the logs do not point to a physical cause.

1.3 Agents built on language models

1.3.1 Principle: supervisor and tools

An *agent*, in the sense used for large language models, is not a model that answers questions but a model given the ability to **act**. The mechanism, known as *tool calling*, consists in describing to the model a set of available functions (their name, purpose and arguments). Faced with a request, the model then produces not text but a structured call: which tool to invoke, with which parameters. The program executes the call and returns the result, on which the model continues its reasoning. A sequence of such calls then leads to an answer to the initial request.

This scheme has a major advantage in a scientific context: the language model computes nothing itself. It invents neither a temperature nor a stress. It decides which program to run and interprets what that program returns. The physics remains entirely inside the solver.

1.3.2 Related work and starting point

The closest work is **AutoFLUKA** [3], published at the end of 2024, which automates the Monte Carlo simulation workflows of the particle transport code FLUKA (a supervisor built on LangChain, domain tools for input file generation, execution and post-processing, a self-healing loop in case of a crash, and a documentary assistant based on semantic search). The motivations behind the present work are the same as those of AutoFLUKA: the learning curve of a scientific computing code is steep, and the workflows are time-consuming and prone to human error.

AutoOFFBEAT therefore reuses the skeleton of AutoFLUKA, substituting the FLUKA parts with OFFBEAT features. Thanks to this help with the implementation, the effort could be concentrated on what is specific to nuclear fuel, namely the physical content of the knowledge bases. Three differences are worth highlighting, as they delimit the two pieces of work.

The **domain**. FLUKA is a transport code; its output quantities are count rates and fluences. OFFBEAT is a thermomechanical code whose outputs are continuous fields comparable to **quantified safety criteria**. This difference makes possible a feature absent from AutoFLUKA: the automatic analysis of safety margins, with its editable threshold base.

Prediction. AutoFLUKA remains reactive: it executes and post-processes. The extension towards the digital twin developed here (chapter 3), in particular the Gaussian-process surrogate, makes it possible to answer questions without re-running the solver.

Evaluation. The literature on such agents essentially reports successful case studies. A significant part of the present work consisted in measuring the limits of the system, from the actual reach of self-healing to the accuracy of tool selection (chapter 4).

1.4 Scope adopted

The scope was fixed as follows:

- the base agent is the priority, and is built **one block at a time**, each tested before moving to the next;
- the code must remain **independent of the language model provider**, so that it can run entirely locally and at no cost;
- the digital twin strand is only started once the base agent is stable;
- the scale adopted is the **single fuel rod**.

Chapter 2

Architecture of the AutoOFFBEAT agent

2.1 Overview

The architecture adopted is that of a single supervisor orchestrating specialised tools (figure 2.1). The user expresses a request in natural language in a web interface. The supervisor, built on a language model, selects and chains the necessary tools. Each tool performs a concrete action and returns a textual report that the model interprets.

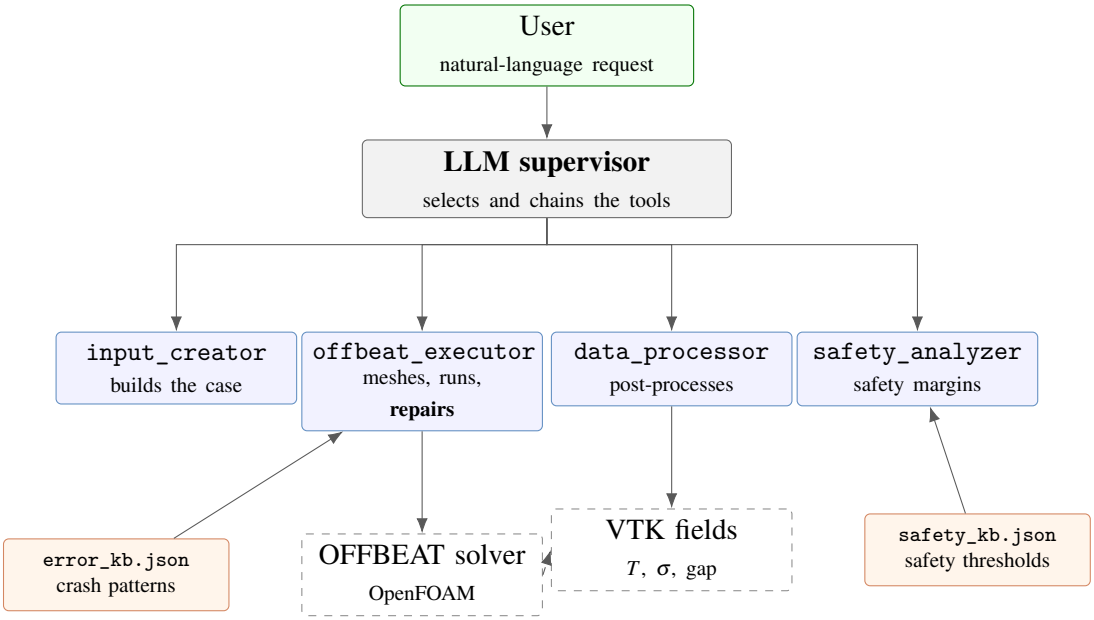


Figure 2.1: Architecture of AutoOFFBEAT.

Three principles underpin this architecture.

The language model does not compute. Every physical quantity comes from the solver or from **deterministic** post-processing. This separation guarantees that the validity of the results does not depend on the reliability of the language model, which could otherwise hallucinate values.

Deterministic first, model as fallback. Wherever a diagnosis can be obtained from an explicit rule (an error pattern, a quantified threshold), that rule takes precedence. The language model only intervenes on failure, and its opinion is never applied automatically.

Domain knowledge lives outside the code. Error patterns and safety criteria are stored in editable JSON files. A domain user can enrich the base without touching the code or rebuilding anything.

2.2 The language model factory

So as not to tie the project to a particular provider, access to the language model goes through a single factory driven by a configuration file. Three providers are supported: fully local execution

through Ollama, and two remote services. The rest of the code is unaware of which one is in use.

This choice had two beneficial consequences. First, the whole development could be carried out **at no inference cost**, with a seven-billion-parameter model running locally. Second, the factory distinguishes the supervisor model from the *debugging* model: it becomes possible to entrust routine routing to a small, fast model and crash analysis to a more capable one.

2.3 The domain tools

2.3.1 Case generation

Generating a complete OFFBEAT case from scratch is not realistic. The solver's main dictionary alone runs to several hundred lines of coupled models, and a language model would inevitably produce invalid combinations. The strategy adopted is therefore that of the **template**: the tool copies a validated reference case, then substitutes only the requested parameters.

A mapping table associates understandable names (linear heat rate, pellet radius, simulated duration) with the actual locations in the case files. This method is essential: it gives the language model a stable and restricted vocabulary, without requiring it to know the syntax of OpenFOAM dictionaries. This matters all the more because a local model with few parameters is used, which can quickly make mistakes. The complete list of exposed parameters is given in appendix Appendix B.

Two templates are currently available: a one-dimensional PWR rod, used for all the results in this report, and a two-dimensional axisymmetric (r, z) rod taken from the OFFBEAT verification cases. The latter was validated end to end (creation, parameter application, meshing, execution and safety analysis), which confirmed that the mesh-region-based processing does not depend on the dimensionality of the case.

2.3.2 Solver execution

The execution tool chains mesh generation and then the solver run, relying on a maximum-duration safeguard so that a diverging simulation cannot block the system indefinitely. It first checks that the target directory really is an OpenFOAM case, and analyses the run log afterwards.

2.3.3 Post-processing

Post-processing reads the solver outputs in VTK format and extracts the useful quantities: axial and radial temperature profiles, hoop stress, maximum cladding temperature. It also produces figures. One subtlety, which turned out to be a significant source of errors (§4.8), lies in the organisation of the data: the complete mesh contains both the pellet and the cladding, and an extremum computed over the whole domain does not necessarily say anything about the region of interest.

2.4 Self-healing

This is the agent's most distinctive contribution. When the solver crashes, the execution tool attempts to diagnose the cause and remedy it. It implements a two-stage strategy, shown in figure 2.2.

The first stage matches the run log against a pattern base. Each entry associates a regular expression with a plain-language diagnosis and a scripted correction (reduce the time step, clean the written time directories to force a clean restart, adjust the simulated duration). This mechanism is fast, free and reproducible.

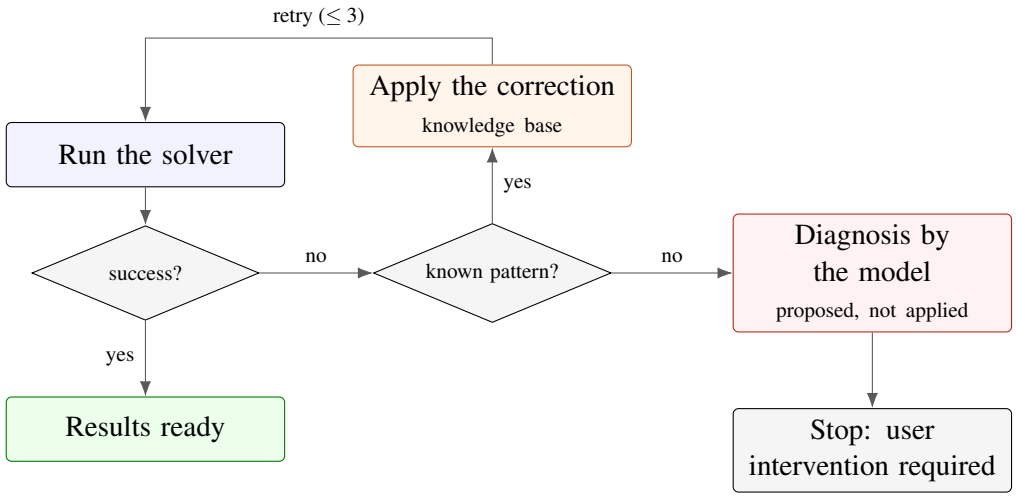


Figure 2.2: Self-healing loop. Deterministic pattern-based diagnosis is attempted first; the language model only intervenes as a last resort and its diagnosis is never applied automatically.

The second stage only intervenes if no pattern matches: the end of the log is then submitted to the language model for interpretation. Crucially, this diagnosis is **presented to the user but never applied automatically**: a human must have the final say on error-sensitive actions.

The number of attempts is capped at three. Chapter 4 shows that this architecture succeeds on the numerical errors it was designed to handle, but reaches its limits when faced with *physical* errors. This is one of the main lessons of the internship.

2.5 The documentary assistant

A final tool, optional and similar to the one in AutoFLUKA, makes the OFFBEAT documentation available to the agent through semantic search (*retrieval-augmented generation*). Documents are split into chunks, embedded by a local embedding model, and indexed. The agent can thereby answer documentation questions by quoting real excerpts rather than producing plausible but unverified statements. This tool is designed to disable itself silently if unavailable: it must never prevent basic operation.

Chapter 3

From reactive agent to digital twin

3.1 Scoping: what this twin is, and what it is not

It is worth clarifying first what is meant by a "digital twin of a power plant". A complete digital twin of a plant would handle the full coupled multiphysics: neutronics, thermal-hydraulics and fuel performance. It would thereby be able to predict the state of a fuel rod from the state of the core, and conversely. OFFBEAT, however, only deals with fuel performance. What is built here is therefore the digital twin of a **single fuel rod**, and everything outside that domain is treated as a boundary condition.

The second limitation concerns "real time". OFFBEAT is a batch solver, one run of which takes from a few seconds to several hours, and there is no sensor measuring the state of a rod inside the core in real time. In practice there is no real time but rather an **update-driven** refresh of the state: at each new operating history, the system re-simulates and updates its prognosis. True real time can only come from a **reduced-order model**, hence block D5.

Table 3.1 summarises the shift.

Table 3.1: Differences between the base agent and the intended digital twin.

	Base agent	**Digital twin**
When	after a completed simulation	at each new piece of data
What	creates, runs, post-processes	+ monitors safety thresholds
Behaviour	reactive (repairs a crash that occurred)	predictive (announces a hazard before it occurs)

3.2 D1 — The safety analyser

The first block reads a completed simulation and assigns, criterion by criterion, a status: safe, close to the threshold, or exceeded. It mirrors exactly the pattern of self-healing — an editable JSON knowledge base, a deterministic treatment, and an optional language model fallback to explain in plain language the criteria that are exceeded.

Each criterion describes the field to read, the reduction to apply (maximum, minimum, largest-magnitude value), the mesh region concerned, the limit and its direction — must the quantity stay below or above the threshold? This last point proved trickier than it appears: gap width is the only criterion whose danger is a value that is *too small*, and this special case was the origin of a defect detected late (§4.8).

It must be stressed that the thresholds currently written in the base are **indicative values**, explicitly flagged as unvalidated. They are published design criteria, not invented values, but their exact magnitude depends on the rod and the burnup: validating them with the supervisor is one of the direct follow-ups to this work.

3.3 D2 — Prognosis

The second block exploits the fact that a simulation writes several time steps. For each non-green criterion, the time evolution of the peak is extracted, a linear trend is fitted, and the instant at which the threshold is crossed is extrapolated.

The cautious rule adopted is to announce nothing when the trend is not moving towards the limit: if the quantity is moving away from it, or is stationary, the prognosis says so explicitly rather than producing a meaningless extrapolation. This extrapolation obviously remains first-order, valid as long as the phenomena involved undergo no regime change.

3.4 D3 — Data assimilation

The third block closes the loop: an operating history file is monitored, its last line representing the current state. At each update, the case is rewritten with the new parameters, the earlier time steps are cleaned, the simulation is re-run and the safety analysis is replayed. It is this cycle that gives meaning to the expression “update-driven”.

3.5 D4 — The dashboard

The web interface was extended with a safety panel: per-criterion gauges, refreshed periodically, showing the current value, the limit and the fraction of the threshold reached. To this is added an interactive slider-based simulator, described below.

3.6 D5 — The surrogate

3.6.1 Motivation and principle

The dashboard highlights a tension: the analysis is instantaneous, but it bears on a simulation that has already been run. Exploring a new scenario — what happens if the power increases by ten per cent? — requires re-running the solver.

Block D5 lifts this limitation through a **reduced-order model** (*surrogate*). A design of experiments is built by crossing two parameters, linear heat rate and irradiation duration. For each combination a complete OFFBEAT simulation is run and three safety quantities are recorded; a statistical model is then fitted to this dataset.

The choice of the sampling domain turned out to be the most consequential decision of this whole block. A first version swept durations of a few thousand seconds, which made the design of experiments inexpensive; section 4.3 shows that this apparent economy in fact invalidated the surrogate for the only question that matters, and that the dataset had to be rebuilt entirely over realistic irradiation durations.

3.6.2 Choice of a Gaussian process

The model adopted is a **Gaussian process** (kriging), with a Matérn kernel. This choice, preferred over polynomial regression, is justified by two properties well suited to the very small number of available points:

- a Gaussian process **interpolates** the training points, which is consistent when they come from a deterministic code rather than from noisy measurements;

- it provides an **uncertainty** at every prediction point. This error bar is propagated all the way to the interface: the user sees not only the predicted value but the confidence that can be placed in it, which is indispensable as soon as a safety computation is replaced by an approximation.

Validation is carried out by *leave-one-out*: each point is predicted by a model refitted on all the others. This is an honest measure of generalisation ability, far more meaningful than a goodness-of-fit coefficient over a few dozen points. The results are presented in the next chapter, where it will be seen that a statistically honest validation can nonetheless mislead if the sampled domain is not itself physically relevant.

Chapter 4

Results and stress-testing

This chapter gathers the results obtained. It is organised in three stages: the physical behaviour reproduced by a reference simulation, the quantitative validation of the surrogate, and then the deliberate stress-testing of the agent's self-diagnosis capabilities.

4.1 Verifying the solver against analytical solutions

Before exploiting the results of the tooled chain, it is worth establishing that the solver on which it rests is itself correct. All the results presented later in this chapter are matters of *internal consistency*: the solver is consistent with itself, the surrogate is consistent with the solver. None confronts the chain with an external truth.

For this purpose OFFBEAT distributes a suite of verification cases, some of which have a **closed-form analytical solution**. Three of them, covering three distinct areas of physics, were run in the environment of this internship and compared with their reference solution (table 4.1).

Table 4.1: Verification of the solver against analytical solutions, run in the internship environment (OpenFOAM v2506, OFFBEAT *master* branch). The indicators are those defined by the verification scripts shipped with the code.

Verification case	Physics exercised	Deviation from analytical
Thick cylinder expansion	Elastoplastic mechanics; radial stress at the inner wall, 150 points compared	1.02 % (max) 0.82 % (mean)
Porosity redistribution	Porosity migration under a thermal gradient	0.064 % ($t = 180\,\mathrm{s}$) 0.16 % ($t = 360\,\mathrm{s}$)
Actinide redistribution	Thermal diffusion of actinides, Clément's analytical solution	5.4×10^{-3} % (Pu) 3.2×10^{-3} % (Am)

Agreement is good on all three cases: at worst one per cent on the mechanical case, which is also the most demanding since it involves a plastification front, and deviations of the order of a thousandth of a per cent on the two fuel physics cases.

This result does not establish the validity of the agent — it establishes that of the solver the agent drives, and therefore that any errors observed downstream are attributable to the tooled chain and not to the computational core. This is precisely the separation needed to interpret the correctness defects of section 4.8: none of them came from OFFBEAT.

Remark. A fourth case in the suite (laser heating) could not be run: it fails while reading its own dictionaries under OpenFOAM v2506. This is an incompatibility between that case and that version of OpenFOAM, independent of the work presented here, but it illustrates how fragile verification suites are across version upgrades.

4.2 Reference simulation over an irradiation cycle

4.2.1 Configuration

The reference case is a PWR rod subjected to a linear heat rate of $25\,\mathrm{kW/m}$ over a simulated duration of 6.3×10^7 s, about two years. The geometry is that of the reference template: pellet radius 4.5 mm, cladding inner radius 4.565 mm, hence an initial gap of 65 µm. Seven states were written during the run.

4.2.2 Thermal evolution

Figure 4.1 shows the evolution of the maximum temperatures of the pellet and of the cladding.

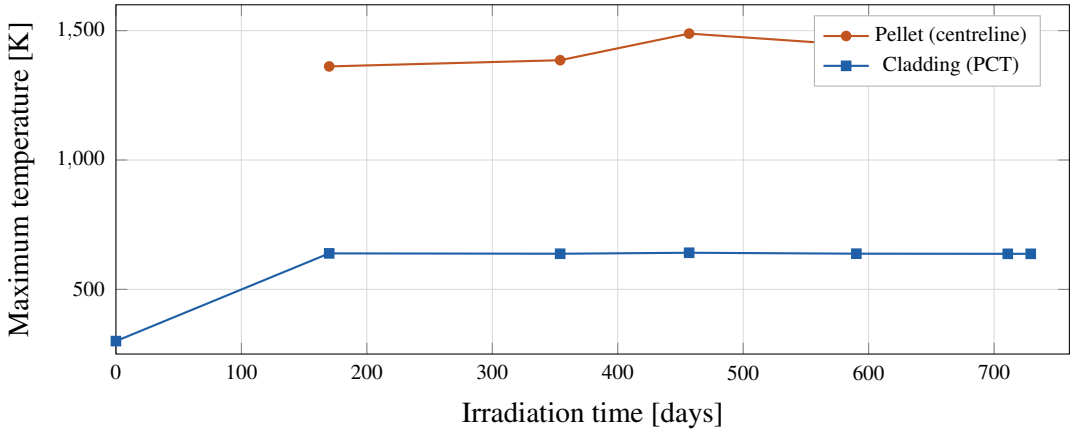


Figure 4.1: Evolution of the maximum pellet and cladding temperatures over the cycle. The gap of about 800 K between the two curves corresponds to the temperature drop across the pellet-cladding gap and the fuel itself.

The cladding temperature remains remarkably stable, around 637 K, which is consistent: it is essentially imposed by the coolant. The pellet centreline temperature, by contrast, rises from 1362 K to 1467 K, reflecting the progressive degradation of the fuel conductivity under irradiation.

4.2.3 Gap closure and stress reversal

The most interesting result appears when the evolution of the pellet-cladding gap is set against that of the hoop stress (figure 4.2).

Reading these two curves together constitutes a qualitative validation of the model's behaviour. As long as the gap remains open, the cladding is **compressed** by the coolant pressure — the stress is negative, of the order of -43 MPa. The gap closes somewhere around day 354 to 457; the stress then becomes **positive** and grows to 79 MPa. This is exactly the signature of PCMI: the pellet, as it swells, puts the cladding into tension. The automatic prognosis (block D2) places the crossing of the closure threshold at about 2.33×10^7 s, that is at one third of the cycle, consistent with the curve.

4.2.4 Spatial profiles

The radial temperature profile at end of cycle (figure 4.3) illustrates the temperature drop across the rod.

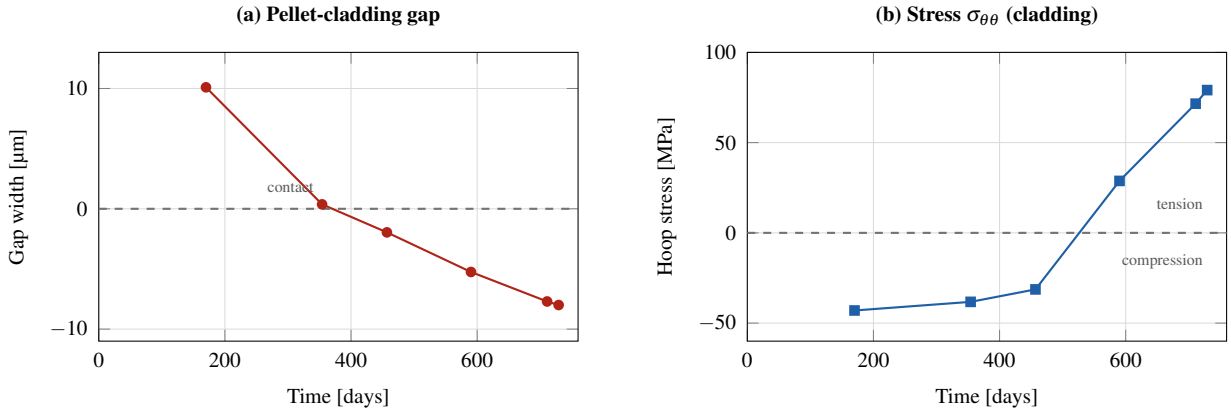


Figure 4.2: Pellet-cladding gap closure (a) and hoop stress in the cladding (b). The change of sign of the stress coincides with the establishment of contact: the cladding, first compressed by the coolant, is then put into tension by the pellet.

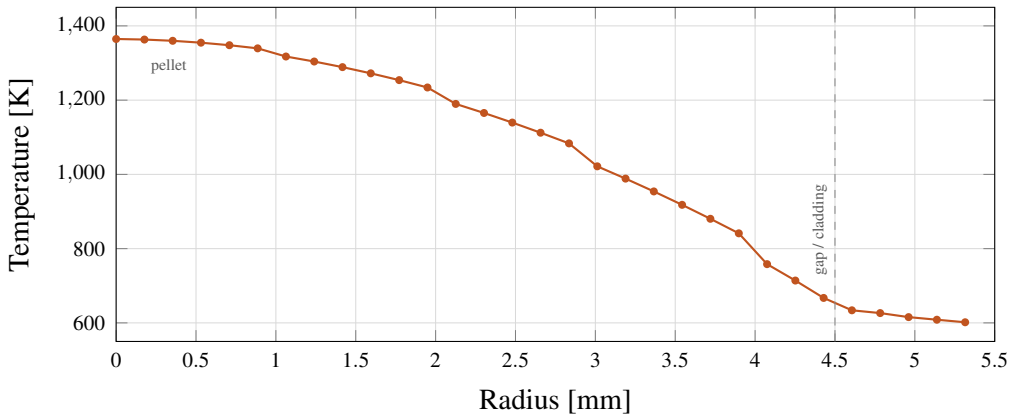


Figure 4.3: Radial temperature profile at the midplane at end of cycle. The drop of about 760 K between the pellet centre (1365 K) and the cladding outer surface (602 K) is dominated by the low conductivity of the fuel and by the thermal resistance of the gap.

4.2.5 Summary of safety margins

Table 4.2 gives the final state as evaluated by block D1.

Table 4.2: Safety margins at end of cycle, after the corrections of section 4.7. The thresholds remain indicative and their validation with the supervisor is still pending.

Criterion	Value	Limit	Status
Pellet centreline temperature	1467 K	3113 K	safe (47 %)
Cladding plastic strain (PCMI)	0	1.00 %	safe (0 %)
Cladding creep strain	0.36 %	1.00 %	safe (36 %)
Cladding hoop stress	79.1 MPa	250 MPa	safe (32 %)
Pellet-cladding gap	−8.0 µm	5.0 µm	**exceeded**

The stress limit here is not a constant: it is read from the `sigmaY` field computed by the solver, and the value reported is the one at the most highly loaded point (section 4.7).

The rod therefore stays comfortably within the margins on all thermomechanical criteria, but pellet-cladding contact is established. This result is expected for a rod at the end of a second cycle and is not alarming in itself: it means that the operating regime has changed in nature, which is precisely the information a digital twin should report.

4.3 Validation of the surrogate

4.3.1 A misleading first training domain

The first version of the surrogate had been trained on sixteen simulations crossing four linear heat rates and four durations between 2000 s to 6500 s. Its *leave-one-out* validation gave coefficients of determination above 0.998 on the three quantities tracked. The result looked excellent.

It was not. Two observations, made by confronting this domain with the reference simulation of the previous section, invalidated it.

First, the rod reaches **thermal equilibrium within a few thousand seconds**. At the two longest durations of the plan, the temperatures differ by only a few tenths of a kelvin (table 4.3): these levels are duplicates. Of the four duration values, only two carried information, and the regression problem was in fact almost one-dimensional — power alone sufficed to predict the output. The high R^2 was therefore mostly measuring how easy the problem was.

Table 4.3: Thermal saturation in the first training domain: beyond 5000 s, the simulated duration no longer carries information.

Linear heat rate	T at 5000 s	T at 6500 s	Difference
12 kW/m	1036.19 K	1036.24 K	0.05 K
20 kW/m	1389.24 K	1389.35 K	0.11 K
28 kW/m	1787.59 K	1787.76 K	0.17 K
36 kW/m	2186.04 K	2186.20 K	0.16 K

Second and above all, the reference simulation runs to 6.3×10^7 s, that is about **ten thousand times** beyond the upper bound of the training domain. Yet the phenomena that matter for safety — swelling, creep, gap closure, PCMI — unfold precisely over these long time scales, entirely absent from the design of experiments. The surrogate was therefore modelling only the

initial thermal transient, while the dashboard's interactive simulator implied that it was predicting safety margins.

4.3.2 Rebuilding over a physically relevant domain

The design of experiments was redone entirely over realistic irradiation durations, from 7.9×10^6 s to 6.3×10^7 s (about three months to two years), crossed with five linear heat rates from 12 kW/m to 36 kW/m, that is 25 complete OFFBEAT simulations, all of which ran to completion.

This rebuild was made possible only by compiling the solver locally: a two-year simulation costs 36 s of computation, so that the complete plan fits in about thirty minutes.

The gain is visible in the data themselves. Over the new domain, the minimum gap width **changes sign**, going from 32 µm to −13 µm depending on the conditions: the surrogate can now learn gap closure, and therefore the onset of PCMI. The circumferential strain also becomes non-monotonic and changes sign, reflecting the inward creep of the cladding before the pellet pushes it back.

Figure 4.4 compares the values predicted by *leave-one-out* cross-validation with those computed by OFFBEAT over this new domain.

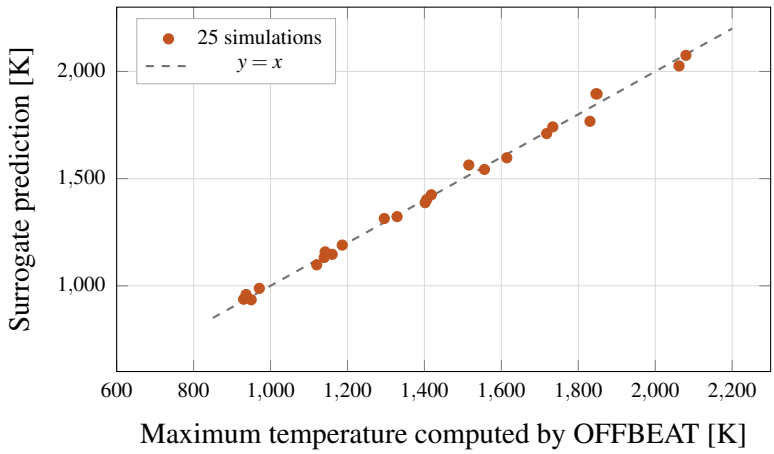


Figure 4.4: Parity plot of the surrogate for the maximum temperature, in *leave-one-out* validation over the realistic irradiation domain. Each point is predicted by a model refitted on the other twenty-four. The coefficient of determination is $R^2 = 0.9951$.

Table 4.4 compares the quality obtained over the two domains.

Table 4.4: Surrogate quality in *leave-one-out* validation over the two training domains. The apparent degradation reflects a problem that has become genuinely two-dimensional and non-monotonic, not a regression of the method.

Predicted quantity	R^2 **(LOO) short domain** (2 ks **to** 6.5 ks)	R^2 **(LOO) realistic domain (3 months to 2 years)**	**Mean error**
Maximum temperature	0.9994	0.9951	19 K
Maximum creep strain	—	0.9905	1.5×10^{-4}
Minimum gap width	0.9992	0.9715	1.8 µm

The coefficients fall, and that is the expected result. Over the short domain the surrogate was fitting a monotonic relation with a single effective variable; over the realistic domain it must

reproduce non-monotonic quantities that change sign. An R^2 of 0.9951 on a hard problem is worth more than 0.9994 on a trivial one. This episode illustrates a general methodological point: **a validation metric says nothing about the relevance of the domain over which it is computed**.

4.3.3 Validation on the safety question

The decisive test does not bear on a coefficient of determination, but on the surrogate's ability to answer the question the digital twin is meant to address: *when does pellet-cladding contact occur?*

The surrogate was therefore queried at 25 kW/m, a power **absent from its training set** (which contains only 12, 18, 24, 30 and 36 kW/m), and its prediction compared with the reference simulation of the previous section (figure 4.5).

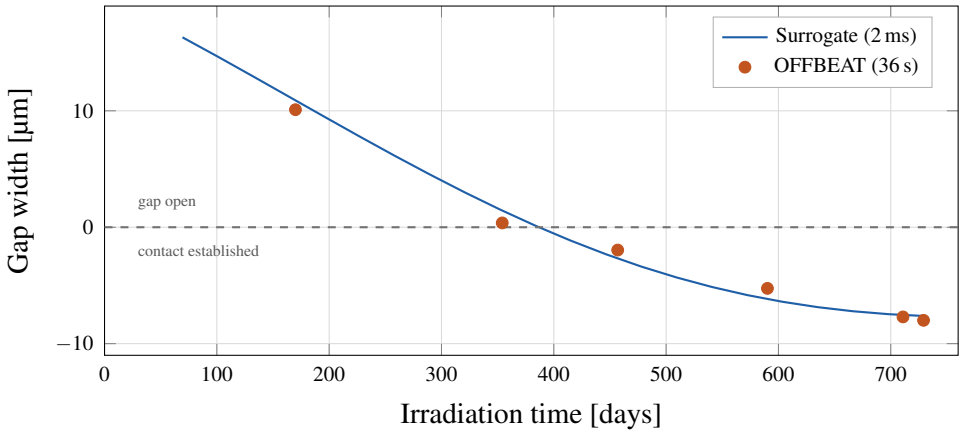


Figure 4.5: Pellet-cladding gap closure at 25 kW/m, a power absent from the surrogate's training set. The continuous curve is the surrogate prediction, the markers are the states computed by OFFBEAT. The zero crossing — the onset of PCMI — is predicted at 387 days against 370 computed, a deviation of 4.4 %.

The surrogate places gap closure at 387 **days**, against 370 **days** obtained by interpolating the states computed by OFFBEAT: a deviation of 4.4 %, that is sixteen days over a two-year cycle. The cost of the prediction is 2 ms, against 36 s for the complete simulation, an acceleration factor of about 18 000.

This result validates the surrogate where it matters: no longer on its ability to reproduce an equilibrium temperature, but on its ability to *date a safety event* by interpolating between operating conditions it has never seen.

Two reservations nonetheless remain. The validation rests on twenty-five points, which is still few; and nothing guarantees validity **outside** the training domain — a surrogate extrapolates poorly by construction, as the first version demonstrated spectacularly. This is precisely why the uncertainty provided by the Gaussian process is systematically reported at the interface alongside the predicted value.

4.4 Stress-testing self-healing

The aim of this section is to delimit, in a measured rather than anecdotal way, what the agent can repair on its own.

4.4.1 Protocol

A benchmark was built on the principle of **fault injection**. A healthy case is first generated by the creation tool, then corrupted in a controlled manner, and finally submitted to the executor with self-healing enabled. Fourteen cases were processed: twelve faults spread over four root-cause categories, plus two healthy cases serving as controls.

The outcome of each case is classified along two deliberately **independent** axes:

- **detected** — did a pattern from the knowledge base recognise the error, as opposed to falling back on the language model;
- **repaired** — did the case eventually run.

Separating these two axes is the point of the protocol: it is the "detected but not repaired" cell that materialises the gap between a *symptom* recognised in a log and a *cause* actually addressed.

Each fault is further labelled a priori according to two distinct criteria: does the base offer a correction for this pattern, and can that correction, by its nature, address the root cause? A fault for which the base *believes* it can intervene while the remedy targets only the symptom is the heart of the problem studied here.

4.4.2 Results

Table 4.5 gives the results by category.

Table 4.5: Self-healing benchmark: 14 cases, of which 12 injected faults. The "stopped at meshing" column marks the cases intercepted by the mesher before the solver was even launched.

Root-cause category	*n*	**detected**	**repaired**	**stopped at meshing**
Control (healthy cases)	2	0/2	**2/2**	0/2
Numerical (solver divergence)	3	3/3	0/3	0/3
Physical consistency (geometry)	3	3/3	0/3	2/3
Missing file	3	1/3	0/3	0/3
Invalid dictionary entry	3	2/3	**1/3**	1/3
Total	**14**	**9**	**3**	**3**

The detection rate is reasonable — nine faults out of twelve are recognised by a pattern, the language model fallback having been called upon only twice. The repair rate, on the other hand, is very low: of the twelve injected faults, **only one** was repaired.

The cross-tabulation of table 4.6 explains why, and constitutes the central result of this section.

Table 4.6: Repair rate depending on whether the available correction can, or cannot, address the root cause of the fault.

Situation	**Repaired**
Correction in the base **and** cause treatable by it	**1/1**
Correction in the base but cause **not** treatable by it	**0/3**
No correction in the base for this pattern	0/8

Self-healing therefore repairs *exactly* the faults whose root cause falls within the scope of the available correction, and only those. The middle row is the most instructive: in those three cases the pattern is recognised, the diagnosis is displayed, the correction fires — and fails every time, consuming the full budget of three attempts each time. The mechanism gives the appearance of working while it structurally cannot succeed.

4.4.3 Analysis of the failures

Three distinct failure mechanisms emerge.

The remedy targets only the symptom. The three numerical faults (powers 100 to 200 times nominal, or negative) trigger a floating-point exception in the creep model. The pattern matches, and the associated correction reduces the time step. But no reduction of the time step can fix a physically absurd input: the cause lies upstream of the solver. The agent therefore applies an ineffective remedy three times in a row.

The correction is undone by the solver. One case is particularly revealing: the time-step reduction is indeed applied and the first computation step succeeds, but OFFBEAT's adaptive time-step controller, seeing that all is going well, immediately increases the step again and causes a new divergence. A robust repair should act on the *maximum* allowed time step, not on the initial one.

The correction converges too slowly for its budget. The benchmark brought to light a defect that one-off trials had not revealed. The correction associated with exceeding the power history reduced the simulated duration by 1 % per attempt. To bring a duration of 1.5×10^8 s below a table ending at 1.26×10^8 s would have required **eighteen** attempts for a budget of **three**: the correction could mathematically never succeed. It was replaced by a deterministic computation, which reads the time tables actually present in the case and targets an admissible value directly. After correction, this case — the only one whose root cause did fall within an available correction — goes from failure to success in two attempts.

4.4.4 A nuance about the "successful" failures

The "stopped at meshing" column of table 4.5 deserves to be read against the grain. Two of the three geometric faults are intercepted by the mesher *before* the solver starts: `blockMesh` refuses to build an impossible geometry. This is not a failure of self-healing, it is a safeguard doing its job. Counting these cases as failures, as a first version of the benchmark did, amounted to penalising the system for correctly rejecting an invalid input — hence the addition of this column.

4.4.5 Lesson

Pattern-based self-healing is suited to errors that are *genuinely* numerical, for which adjusting a resolution parameter makes sense. It is structurally unsuited to **physical consistency** errors and to **configuration** errors, which account for eleven of the twelve faults in this benchmark.

The practical consequence is a shift of effort: rather than endlessly enriching the base of crash patterns, it is more profitable to **validate the inputs before execution**. A case that cannot be physically valid should never reach the solver. A first geometric validation was added to that end (§4.8).

4.5 Evaluating the decision layer

The preceding sections measure the physical chain and the repair loop. One link remained unevaluated, yet central: faced with a request written in natural language, does the supervisor choose the right tool?

4.5.1 Protocol

Twenty-six requests were written, covering the six tools exposed to the agent, plus four requests for which *no* tool should be called — a greeting, a general-knowledge question, a note of thanks. Triggering a tool on these is a false positive just as much as a wrong choice is.

The tools are not executed. They are bound to the model, and it is the *intent* produced — the list of requested calls — that is recorded. This protocol measures selection exactly, with no side effects: no case is created, no solver is launched. It takes one pass per request.

It should be stressed that this evaluation was not practicable before the hardware acceleration described in section 4.9. At four minutes per invocation, these twenty-six requests would have taken almost two hours; they now run in under a minute, which allows several iterations within a single working session.

4.5.2 Results and diagnosis

The first campaign gives an overall accuracy of 77 %. The breakdown by tool, however, is very uneven (table 4.7) and points to a single culprit: the safety analyser is correctly selected in only one case out of four.

Table 4.7: Tool-selection accuracy, before and after rewording the descriptions. A case counts as correct if the expected tool is called first, or if no tool is called when none is expected.

Expected tool	*n*	**Before**	**After**
Case creation	4	4/4	4/4
Solver execution	4	4/4	4/4
Post-processing	4	3/4	3/4
Safety analysis	4	**1/4**	**4/4**
Surrogate	3	2/3	2/3
Documentation	3	3/3	3/3
No tool expected	4	3/4	3/4
Total	**26**	**20/26** (77 %)	**23/26** (88 %)

Examining the three failures is enlightening. On “check whether a safety criterion is exceeded”, the agent calls the executor; on “analyse the PCMI margins and tell me when the limit will be reached”, it calls the surrogate; on “is there a risk of fuel melting?”, it calls nothing.

The cause is not the model, but the descriptions supplied. Those of the safety analyser and of the surrogate promised exactly the same thing — safety margins, per-criterion status, PCMI, gap closure — without ever stating what really distinguishes them. Yet that criterion is simple and unambiguous: **the analyser requires an already-simulated case on disk, whereas the surrogate takes only numerical parameters**.

4.5.3 Correction and re-measurement

The two descriptions were rewritten to state this condition of use explicitly, and to name each other as alternatives not to be confused. No other change was made — neither to the model, nor to the tool code, nor to the requests.

The accuracy of the safety analyser rises from 25 % to 100 %, and the overall accuracy from 77 % to 88 %.

The lesson goes beyond this particular case. In a tool-based architecture, a tool description is not documentation: it is the *programming interface* through which the model decides. Two tools whose descriptions overlap are, from the model's point of view, indistinguishable — and no amount of model quality compensates for that ambiguity. It is also good news in terms of cost: eleven accuracy points were gained by rewriting two paragraphs, with no retraining and no change of model.

The three remaining errors do not stem from this cause. Two concern wordings where the boundary is genuinely debatable — "quickly estimate the maximum temperature" may legitimately call either the surrogate or post-processing — and the third is a documentation call on a general-knowledge question, a benign false positive.

4.5.4 From selection to execution: a blind spot

The evaluation above measures which tool the agent *chooses*. It says nothing about what happens when that call is actually *executed*, since the protocol reads the intent without ever triggering it. That choice, justified in order to isolate the decision, turned out to be a blind spot: the first attempt at end-to-end execution failed completely, on a defect invisible to the selection benchmark.

A tool schema incompatible with actual use. The creation tool declared its `params` parameter as a *string* containing JSON. Yet a language model spontaneously produces an *object* for an argument so named. Validation rejected it, the agent received an error message, and replayed the identical call. Measured: 30 tool calls, 53 messages, 179 s, **no case created at all**. Accepting both forms removed the loop.

A context window that was too narrow. A second defect manifested itself as freezes on multi-step requests, while a simple question was answered in one second. Measurement showed that the system prompt and the tool schemas alone consume ∼1900 **tokens, that is** 47 % **of the** 4096**-token window** used by default — before any exchange. A single tool result was then enough to overflow it. The window was raised to 16 384 tokens.

Table 4.8 gives the combined effect of these two corrections on the same request.

Table 4.8: End-to-end execution of a four-step request (create, simulate, post-process, analyse safety), before and after correcting the tool schema and the context window.

Indicator	**Before**	**After**
Tool calls	30	5
Messages exchanged	53	11
Total duration	179 s	39 s
Case created and simulated	**no**	**yes**

The methodological lesson goes beyond these two fixes. A benchmark delimits what it can see: measuring selection gave 88 % success while execution of the main task was failing in 100 % of cases. **A flattering indicator on one link says nothing about the chain.** The complete

transcript of the successful session is given in appendix Appendix G.

4.6 Evaluating the documentary assistant

The documentary assistant was the last component of the system not supported by any measurement. Its evaluation produced the sharpest result in this whole chapter.

4.6.1 Protocol

What determines the quality of a documentary answer is the **retrieval** that precedes it: if the wrong excerpt comes up, the answer will be wrong whatever the quality of the language model. It is therefore retrieval alone that is measured here.

Fifteen questions were written, each associated with the corpus document that actually contains the answer. Two indicators are recorded: is the right document the source of the first excerpt (accuracy@1), and does it appear among the four retrieved excerpts (recall@4)? The second is the more relevant in practice, since the agent receives all four excerpts.

As the corpus contains only four documents, a random draw would already give about 33 % accuracy@1. This threshold serves as a lower reference.

4.6.2 A result below chance

The first campaign gives 27 % accuracy@1 — that is, **worse than a random draw**. But the truly alarming indicator lies elsewhere: recall@4 is also 27 %, exactly the same value. This equality is the sign of a structural failure. With four excerpts retrieved and four documents in the corpus, recall@4 ought to be close to 100 %. That it equals accuracy@1 means the four excerpts *always come from the same document*: retrieval systematically collapses onto a single source.

4.6.3 Diagnosis: a cross-lingual failure

Examining the corpus provides the explanation. It is **bilingual**, and the split is clear-cut: the two files that actually document OFFBEAT — the structure of a case and the list of commands — are in **English**, since they come from the code repository; the documents added by the project are in **French**.

Yet the agent is driven in French. The embedding model in use then matches French questions to French documents, irrespective of their content. The OFFBEAT documentation proper is never reached.

The control experiment removes all doubt: the same questions asked in English correctly retrieve the English documents.

Table 4.9: Effect of the question language on documentary retrieval, with corpus and model unchanged. The retrieval mechanism works; it is crossing the language barrier that fails.

Question language	**Accuracy@1**	**Recall@4**
French → English documents	**0/11**	**0/11**
English → English documents	9/11 (82 %)	**11/11** (100 %)

4.6.4 Correction and re-measurement

The natural fix is not to translate the documentation — it belongs to the upstream repository and would have to be maintained at every version upgrade — but to replace the embedding model with a **multilingual** one, which projects a French text and its English translation to the same place in the vector space. The corpus and the index were rebuilt identically, only the embedding model changing.

Table 4.10: Documentary retrieval quality before and after switching to a multilingual embedding model. Corpus, questions and chunking identical.

Indicator	**Initial model**	**Multilingual model**
Accuracy@1	27 %	87 %
Recall@4	27 %	100 %
Case structure	0/6	**6/6**
Commands	0/5	3/5 (recall@4: 5/5)
Adding an error entry	4/4	4/4

Recall@4 reaches 100 %: the agent now always receives the right document among its four excerpts. The multilingual model has become the project's default.

This case completes the two preceding ones and forms a pattern with them: none of these three failures — overlapping tool descriptions, degenerate training domain, cross-lingual retrieval — was visible without measurement, and each was fixed without rewriting any application code. What was missing was not technique, but *instrumentation*.

4.7 Confronting the criterion with what the solver computes

In the architecture adopted, safety thresholds are simple constants written in a JSON file. Checking these constants against the physical model actually solved revealed two defects of the same nature, and suggested a deeper improvement.

4.7.1 A criterion checking a limit the simulation does not use

The cladding yield strength was set to 350 MPa in the criteria base. Yet the template configures the plasticity model with a limit of 250 MPa. The safety criterion was therefore evaluating the stress computed by the solver *against a limit that the very same solver does not apply* — a discrepancy of 100 MPa, that is 40 %. On the reference case, the reported margin thus went from 32 % in reality to 23 % as displayed.

The fix does not consist in copying across the right constant, since that would reproduce the same risk of drift. OFFBEAT writes the yield strength as a **field**, `sigmaY`, cell by cell: the criteria base was therefore extended so that a limit may designate a field (`limit_field`) rather than a constant. The ratio to the limit is then computed *point by point* and only then reduced:

$$\max_{x} \frac{|\sigma_{\theta\theta}(x)|}{\sigma_Y(x)}$$

This point-by-point computation is not a cosmetic refinement. The stress peak and the minimum of the limit are not at the same location; comparing the maximum of one with the minimum of the other yields a result that corresponds to no point of the rod. On a test case built for

the purpose, the naive criterion concludes 83 % of the limit where the correct computation gives 150 % — that is, a missed exceedance.

The main benefit is architectural: the criterion now *automatically* inherits any temperature, irradiation or burnup dependence that the material model may come to carry, without a single value having to be maintained twice.

4.7.2 A criterion measuring something other than what it claimed

The PCMI criterion states a limit on the **plastic** circumferential strain. It was in fact reading `epsilonCyl`, the **total** strain. The decomposition of the strains on the reference case (table 4.11) shows the extent of the misunderstanding.

Table 4.11: Decomposition of the circumferential cladding strain at end of cycle. The PCMI criterion was reading the total, whereas its statement bears on the plastic component alone — which is exactly zero.

Component	**Maximum value**	**Reversible?**
Total (`epsilonCyl`, read by the old criterion)	1.81×10^{-3}	—
Plastic (`epsilonP`)	**exactly 0**	no
Creep (`epsilonCreepEq`)	3.55×10^{-3}	no
Elastic (`epsEl`)	1.03×10^{-3}	yes
Thermal (`epsTh`)	2.31×10^{-3}	yes

The criterion was therefore reporting 18 % of a limit whose measured quantity is in reality at 0 %. The error erred on the conservative side — total strain bounds plastic strain from above — but it measured the wrong quantity, and above all it masked the essential point: the irreversible strain actually present is the **creep** strain, almost twice the reported total, and no criterion was monitoring it.

The PCMI criterion now reads the plastic strain, in accordance with its statement, and a separate criterion was added for creep strain. Whether the 1 % design criterion bears on plasticity alone or on total permanent strain remains an open question and is among the points to be settled with the supervisor.

4.7.3 Knock-on effect on the surrogate

These corrections had a consequence that only a systematic check could reveal. The surrogate of chapter 3 records its target quantities *through* the criteria base: making the PCMI criterion conform to its statement therefore brought one of its three targets down to zero, plastic strain being null in nominal operation. The target would have been degenerate, with no variation to learn.

The surrogate now targets creep strain, and the design of experiments was rebuilt accordingly. The outcome is better than before: creep accumulates monotonically, where the total strain was non-monotonic, and the cross-validation coefficient of determination rises from 0.9535 to **0.9905** (table 4.4). The correct quantity turns out to be the easier one to learn as well. The prediction of the gap closure date, the main result of section 4.3, is unchanged.

This is one more illustration of the coupling between components believed to be independent: a correctness fix in the threshold base silently invalidated a learning target two layers away. It was detected only because every change was followed by an end-to-end check.

4.8 Correctness defects identified and fixed

A second form of stress-testing proved at least as fruitful: no longer provoking crashes, but confronting obtained results with expected ones. Fourteen defects were identified in this way, **none of which raised any visible error** (table 4.12). These are *correctness* errors, silent by nature, and therefore particularly worrying in a safety context: the system was producing plausible, correctly formatted, and wrong numbers.

The two defects concerning cladding quantities — the maximum temperature and the hoop stress — share a common origin, worth spelling out because it is instructive. The mesh of a rod contains both the pellet and the cladding. Taking the maximum of a field "over the domain" therefore amounts, for temperature, to measuring the pellet centreline — the hottest region — whereas the criterion bears on the cladding. The initial code was aware of this and documented the choice as a conservative approximation; the analysis showed that it was not always so, since in the case of stress it reversed the sign of the result.

The fix consisted in exploiting the *cell zones* defined by OFFBEAT, which explicitly distinguish fuel from cladding in the mesh. This required enabling a reader option that is not active by default. The fix falls back silently to the previous behaviour if the zone is unavailable, so that no existing case can be broken by the change.

4.9 Performance of the execution environment

Porting the project to a new machine was the occasion for a notable performance gain on language model inference, which was the main brake on development. Table 4.13 gives the measurement.

The acceleration factor of about 190 between CPU and GPU execution changes the working conditions qualitatively: it becomes possible to test the agent in full loop rather than resorting to workarounds to avoid waiting. The difficulties encountered in obtaining this acceleration are related in the next chapter.

Table 4.12: Correctness defects identified during use and subsequently fixed.

Component	Symptom	Cause and scope
Safety analyser	A closed, hence negative, pellet-cladding gap was reported as safe	The ratio to the limit is computed by division; a negative value reverses its sign. The only exceeded criterion of the reference case was masked
Post-processing	The "maximum cladding temperature" read 1467 K	The extremum was taken over the whole mesh, hence at the pellet centreline. A 830 K discrepancy with the true value (637 K)
Safety analyser	Cladding stress and strain overestimated	Same cause; the sign itself was wrong, tension being reported where the cladding is in compression
Case generator	Geometry file corrupted after modifying a multi-valued parameter	The substitution regular expression stopped at the first comma of a list, producing an invalid literal
Case generator	No geometric safeguard	Pellet and cladding radii never compared with one another (see §4.4)
Surrogate	*Leave-one-out* validation at $R^2 > 0.998$	Degenerate training domain: thermal saturation from 5000 s onwards, and 10 000 times below the durations of interest (§4.3)
Self-healing	The simulated-duration correction never succeeded	1 % decrement per attempt: 18 attempts needed for a budget of 3 (§4.4)
Tool descriptions	The safety analyser was selected only once in four	Overlap with the surrogate description, without stating the distinguishing criterion (§4.5)
Field reading	All criteria reported "field absent" on a multi-region case	Merging the mesh blocks includes the regions without data and loses the fields; detected by carrying the tools over to the verification cases (§4.1)
Documentary assistant	Retrieval performing worse than a random draw	Bilingual corpus and monolingual embedding model: French questions never reached the English documentation (§4.6)
Creation tool schema	The agent looped without ever creating a case	The parameter was declared as a JSON string where a model produces an object; 30 calls without success (§4.5.4)
Context window	Freezes on multi-step requests	4096 tokens by default, 47 % of which consumed by the system prompt and the tool schemas before any exchange (§4.5.4)
Cladding stress threshold	The criterion checked 350 MPa while the solver applies 250 MPa	Frozen constant, out of sync with the material model; margin displayed as 23 % instead of 32 % (§4.7)
PCMI criterion	Reported 18 % of a plastic-strain limit	Read the *total* strain; the plastic part is exactly zero, and creep — irreversible — was not monitored (§4.7)

Table 4.13: Latency of a single language model invocation (7 billion parameters, short request).

Configuration	**Latency**
CPU only	64 s
GPU, first call (model loading)	11.4 s
GPU, model already loaded	0.33 s

Chapter 5

Difficulties, false starts and schedule

This chapter reports the obstacles encountered. It is not a catalogue of anecdotes: each of these difficulties changed a design decision or a working method.

5.1 Technical difficulties

5.1.1 Hardware acceleration of the language model

Development was initially slowed by agent invocations exceeding four minutes, the language model running on the CPU. As the working machine had a capable graphics card, exploiting it seemed straightforward. This was the longest-standing difficulty to resolve, for three linked reasons.

First, the usual acceleration software stack for that graphics card vendor does not cover the model available on the machine under the Linux environment used. It had to be identified that an alternative route, based on a standard graphics interface, was available and should be preferred.

Second, a misleading error message cost time: the log stated that the graphics driver was too old, whereas it was up to date. That message in fact came from a check relating to the stack that had not been selected, unrelated to the route actually in use. The lesson is a classic one but worth retaining: an explicit error message is not necessarily a *relevant* error message.

Third, a second installation of the same service, made by mistake inside the Linux environment instead of the host system, was silently intercepting every request. The measurements obtained were therefore those of the slow configuration, even though the fast one was in place. This confusion illustrates a methodological point: when two equivalent services share a port, no measurement is reliable until one has checked *which* of them answers.

5.1.2 Compiling the solver

Installing OFFBEAT ran into a simple obstacle: the repository address given in the project's internal documentation was no longer valid. The actual public repository had to be identified, and its build structure turned out to differ from the one described. Compilation itself then proceeded without error.

This episode led to a more general remark: a project's documentation ages, and wrong documentation costs more than absent documentation, because it actively points in the wrong direction.

5.1.3 Undeclared dependencies

Porting to a fresh machine revealed three Python libraries used by the code but absent from the dependency list. They were present by accident on the original machine. This kind of defect only shows up during a transfer, which argues for performing such a transfer *early* rather than at the end of a project.

5.2 False starts and reorientations

5.2.1 A correct but ineffective diagnosis

The most instructive false start concerns the very design of self-healing. The initial assumption, inherited from the reference project, was that a crash is sufficiently characterised by its signature in the run log. The stress-testing of chapter 4 showed that this assumption is insufficient: two crashes with identical signatures may call for opposite remedies, or for no automatable remedy at all.

The resulting reorientation is a shift of effort: rather than endlessly enriching the base of crash patterns, it is more effective to **validate inputs before execution**. A case that cannot be physically valid should never reach the solver.

5.2.2 A conservative approximation that was not

The second false start is the initial choice to measure cladding quantities over the whole domain, documenting it as a conservative approximation. The reasoning seemed sound: overestimating a stress can only err on the side of safety.

It was wrong. In the case of hoop stress, the extremum retained lay inside the pellet and carried a sign opposite to the actual stress in the cladding — tension reported where the cladding was in compression. An approximation prudent in principle had turned into a qualitatively false result.

The lesson drawn goes beyond this particular case: a choice documented as conservative must be *verified* to be so, not assumed. The correction was validated on a real case, where it did change the strain result by a factor of five and reversed the sign of the stress.

5.2.3 A rigorous validation of an uninteresting problem

The third false start is the one from which I learned the most, because it concerned not a bug but a *method*.

The surrogate had been validated with what seemed to me every precaution: *leave-one-out* cross-validation, chosen precisely because it is more honest than a goodness-of-fit coefficient, and coefficients of determination above 0.998 on the three quantities tracked. The statistical method was correct and the result excellent.

The problem is that the question asked was not. The sampling domain — a few thousand seconds — had been chosen so that the design of experiments would remain quick to build, at a time when the solver was not compiled locally. Yet over that domain the rod reaches thermal equilibrium: the irradiation duration, the model's second variable, carried almost no information. I had therefore rigorously validated a model on an almost one-dimensional problem, while the dashboard was presenting its predictions as two-parameter safety margins.

What made this visible was not a metric but a confrontation: the reference simulation ran to 6.3×10^7 s when the surrogate had never seen beyond 6500 s. A ratio of ten thousand between the training domain and the domain of use cannot be read off an R^2.

The lesson is twofold. On the one hand, a validation metric only qualifies the fit to a dataset, never the relevance of that dataset: a model can be impeccably validated and nonetheless useless. On the other hand, the practical constraint that had dictated the choice — the cost of building the design of experiments — had disappeared once the solver was compiled locally, without my re-examining the decision it had motivated. Design assumptions deserve to be re-evaluated when the constraints that justified them change.

5.2.4 On the use of a programming assistant

Development was carried out with the support of a programming assistant based on a language model. The perspective gained is worth recording, since the subject of the internship concerns this very class of tools.

The assistance proved very effective for producing structured code and for exploring an unfamiliar codebase. It proved, on the other hand, incapable of detecting on its own the correctness defects of table 4.12: these were found only by *confronting numerical results with a physical expectation* — why is the cladding temperature equal to the pellet centreline temperature? why is a closed gap reported as safe? Physical judgement remains the decisive link, and this observation applies equally to the agent developed here.

Internship schedule

[To be adjusted to the actual internship dates. The phases below reflect the order in which the work was actually built.]

Table 5.1: Progress of the internship by phase.

Period	Phase	Deliverable / milestone
Weeks 1–2	Familiarisation	Fuel physics, first steps with OpenFOAM and OFFBEAT, study of the reference project AutoFLUKA
Weeks 3–4	Software foundation	Project tree, provider-agnostic language model factory, development environment
Weeks 5–6	Execution and self-healing	Solver execution tool validated on a real case; error pattern base and repair loop
Weeks 7–8	Complete agent	Supervisor, case generation tool, post-processing; first end-to-end chaining
Weeks 9–10	Interface and documentation	Web interface, documentary assistant based on semantic search
Weeks 11–13	Digital twin	Safety analyser, prognosis, assimilation, dashboard, Gaussian-process surrogate
Weeks 14–15	Validation and consolidation	Reference simulation over a complete cycle, stress-testing of self-healing, correction of the correctness defects
Week 16	Writing up	Report and defence

Conclusion

Summary of the work

This research project has resulted in a working system, **AutoOFFBEAT**, which makes it possible to drive the fuel performance code OFFBEAT from requests expressed in natural language. The agent creates computational cases from validated templates, runs the solver, attempts to repair certain crashes automatically, post-processes the results and evaluates safety margins against design criteria.

The system was then extended towards a fuel rod digital twin: monitoring of safety criteria, prognosis of threshold crossing, dashboard, and a Gaussian-process surrogate returning the results of a simulation within a few milliseconds, with an associated uncertainty and a fidelity verified by cross-validation.

A reference simulation over a two-year irradiation cycle made it possible to verify that the complete chain reproduces the expected physical behaviour, in particular the closure of the pellet-cladding gap and the associated switch of the cladding stress from compression to tension. The surrogate, rebuilt over a realistic irradiation domain, predicts the date of that closure to within 4.4 % for a power it has never encountered, in 2 ms instead of 36 s.

The second strand of the work, less expected at the outset, is the **measured stress-testing of the system**. A fault-injection benchmark quantified the actual reach of self-healing; an evaluation of the decision layer measured, then improved by eleven points, the accuracy of tool selection. Both benchmarks are re-runnable and delivered with the code: they constitute a non-regression harness for the rest of the project.

What I learned

Three lessons seem to me worth retaining.

The first is a matter of physics. I was discovering fuel performance entirely at the start of the internship. Understanding that the pellet-cladding gap governs both the thermal and the mechanical behaviour of the rod, and being able to read the signature of contact in simulation curves, is the clearest technical gain.

The second concerns the software architecture of systems built on language models. The principle that structures the whole project — the model orchestrates, it does not compute; deterministic first, model as fallback — carries well beyond OFFBEAT.

The third is methodological, and is perhaps the most important. The most dangerous defects encountered were not crashes but wrong results that did not look wrong. No automatic tool flagged them; they were found by confronting a numerical value with a physical expectation. A safety computation chain is not validated by checking that it runs, but by checking that what it produces makes sense.

This third point admits a corollary I had not anticipated: a validation metric can itself be misleading. The surrogate displayed a coefficient of determination above 0.998, obtained by a rigorous method, over a domain that did not cover the real problem. Checking that a model is well fitted and checking that it is fitted to the right question are two distinct operations, and only the

second falls to physical judgement.

Outlook

Several concrete follow-ups stand out, in order of priority.

Validate the safety thresholds. The criteria currently written into the knowledge base are marked as unvalidated. Confirming them with the supervisor and the literature is the prerequisite to any serious use of the safety analysis.

Settle the scope. Moving from the single rod to the assembly changes the nature of the problem and remains an open question.

Shift the effort from repair to validation. The analysis of chapter 4 shows that validating inputs before execution is more reliable than enriching the base of crash patterns. A first geometric validation was added; it could be extended systematically to all the exposed parameters.

Extend the surrogate. The surrogate covers only two parameters. Since its reconstruction over a realistic domain showed that a design of twenty-five simulations fits in half an hour, adding a third variable — the initial gap geometry, for instance — is now within reach. The code is written for an arbitrary number of variables.

Make time-step self-healing reliable. The benchmark showed that the fix is undone by the solver's adaptive controller; acting on the maximum time step rather than on the initial one is an identified and localised correction.

Complete the validation of the thresholds. The correlation for the decrease of the melting temperature with burnup was settled at the end of the internship by returning to the literature: the value carried by the project came from the MATPRO correlation, since superseded in the reference codes (appendix Appendix D). The same approach remains to be carried out on the three other criteria, including the yield strength of Zircaloy, strongly temperature-dependent and for the moment given as a single value.

Extend the tool-selection benchmark. The current twenty-six requests were enough to reveal a major ambiguity between two descriptions. A broader set, and above all multi-step requests, would allow the chaining of tools to be evaluated rather than the first call alone.

Bibliography

[References to be completed and checked — in particular the reference papers on OFFBEAT, to be cited in their published version.]

Journal articles

[1] SCOLARO, Alessandro, CLIFFORD, Ivor, FIORINA, Carlo, PAUTZ, Andreas. The OFFBEAT multi-dimensional fuel behavior solver. *Nuclear Engineering and Design*, 2020, vol. 358, 110416. *[references to be checked against the published version]*

[2] SCOLARO, Alessandro *et al.* Extension of the OFFBEAT fuel performance code to finite strains and validation against LOCA experiments. *Nuclear Engineering and Design*, 2023. *[volume and pagination to be completed]*

[3] NDUM NDUM, Zavier, TAO, Jian, FORD, John, LIU, Yang. AutoFLUKA: A Large Language Model Based Framework for Automating Monte Carlo Simulations in FLUKA. *arXiv preprint* arXiv:2410.15222, 19 October 2024. DOI 10.48550/arXiv.2410.15222. *[a journal version exists in Energy and AI (2025); check volume and article number before submission]*

[4] GEELHOOD, Kenneth J., LUSCHER, Walter G. FRAPCON and FRAPTRAN codes: fuel rod performance analysis codes under normal and accident conditions. In: *Advances in Nuclear Fuel Chemistry*. Elsevier, 2020. *[pagination to be completed]*

[5] LASSMANN, Klaus. TRANSURANUS: a fuel rod analysis code ready for use. *Journal of Nuclear Materials*, 1992, vol. 188, p. 295–302. *[reference to be checked]*

[6] WILLIAMSON, Richard L. *et al.* Multidimensional multiphysics simulation of nuclear fuel behavior. *Journal of Nuclear Materials*, 2012, vol. 423, no. 1-3, p. 149–163. *[reference to be checked]*

Technical reports and safety standards

[7] PACIFIC NORTHWEST NATIONAL LABORATORY. *Pellet-Cladding Mechanical Interaction Failure Threshold for Reactivity Initiated Accidents*. Report PNNL-22549. `https://www.pnnl.gov/main/publications/external/technical_reports/PNNL-22549.pdf` (accessed *[date]*).

[8] U.S. NUCLEAR REGULATORY COMMISSION. *Pellet Cladding Interaction Fuel Failures during Anticipated Operational Occurrences*. Document ML14107A179. `https://www.nrc.gov/docs/ML1410/ML14107A179.pdf` (accessed *[date]*).

[9] INTERNATIONAL ATOMIC ENERGY AGENCY. *Technical and economic limits to fuel burnup extension*. IAEA-TECDOC-1299, Vienna, 2002. `https://www-pub.iaea.org/MTCD/Publications/PDF/te_1299_web.pdf` (accessed *[date]*).

[10] U.S. NUCLEAR REGULATORY COMMISSION. *10 CFR 50.46 — Acceptance criteria for emergency core cooling systems for light-water nuclear power reactors. [to be checked and completed]*

[11] LUSCHER, Walter G., GEELHOOD, Kenneth J., PORTER, Ian E. *Material Property Correlations: Comparisons between FRAPCON-4.0, FRAPTRAN-2.0, and MATPRO*. Report PNNL-19417 Rev. 2, Pacific Northwest National Laboratory, prepared for the U.S. Department of Energy, September 2015. § 2.1 (*Fuel Melting Temperature*, subroutine PHYPRP).

Websites and technical documentation

FOAM-FOR-NUCLEAR. *OFFBEAT Documentation* [online]. `https://foam-for-nuclear.gitlab.io/offbeat/` (accessed *[date]*).

FOAM-FOR-NUCLEAR. *OFFBEAT source repository* [online]. `https://gitlab.com/foam-for-nuclear/offbeat` (accessed *[date]*).

OPENCFD LTD. *OpenFOAM v2506 User Guide* [online]. `https://www.openfoam.com/documentation/` (accessed *[date]*).

SMARTLABNUCLEAR. *AutoFLUKA* [online]. `https://github.com/SmartLabNuclear/AutoFLUKA` (accessed *[date]*).

LANGCHAIN. *LangChain / LangGraph documentation* [online]. `https://python.langchain.com/` (accessed *[date]*).

Glossary

English	French	Definition
Burnup	Taux de combustion	Energy extracted per unit mass of fuel; a measure of ageing
Cladding	Gaine	Metallic tube (Zircaloy) surrounding the fuel
Coolant	Caloporteur	Fluid removing heat from the core
Creep	Fluage	Slow deformation under constant stress
Data assimilation	Assimilation de données	Update of a model from new operating data
Densification	Densification	Volume reduction of the fuel early in irradiation
Digital twin	Jumeau numérique	Numerical model of a physical system, kept up to date by its operating data
DNBR (Departure from Nucleate Boiling Ratio)	—	Margin to the boiling crisis; a thermal-hydraulics quantity
Finite volume method	Volumes finis	Conservative discretisation method
Fission gas release (FGR)	Relâchement des gaz de fission	Release of fission gases out of the fuel matrix
Fuel rod	Crayon combustible	Cladding tube containing the column of fuel pellets
Gaussian process	Processus gaussien	Regression method providing a prediction together with its uncertainty
Hoop stress	Contrainte de cerclage	Circumferential component ($\theta\theta$) of the stress tensor in the cladding
Leave-one-out cross-validation	Validation croisée « un contre tous »	Validation in which each point is predicted by a model fitted on the others
LLM agent	Agent (LLM)	System in which a language model selects and chains software tools to accomplish a task
Mesh	Maillage	Subdivision of the domain into computational cells
PCMI (Pellet-Cladding Mechanical Interaction)	Interaction mécanique pastille-gaine	Mechanical loading of the cladding by the pellet after gap closure
PCT (Peak Cladding Temperature)	—	Maximum temperature reached by the cladding
Pellet	Pastille	Cylinder of sintered uranium oxide
Pellet-cladding gap	Jeu pastille-gaine	Initial radial space between pellet and cladding
Prognosis	Pronostic	Prediction of the instant at which a threshold is crossed
PWR (Pressurized Water Reactor)	REP	Pressurised water reactor
RAG (Retrieval-Augmented Generation)	—	Generation assisted by documentary retrieval

English	**French**	**Definition**
Self-healing	Auto-réparation	Automatic diagnosis and correction of a computation crash
Solver	Solveur	Program solving the equations of the model
Surrogate model	Émulateur / modèle réduit	Fast statistical model approximating the outputs of an expensive computational code
Swelling	Gonflement	Volume increase of the fuel under irradiation

Appendices

Appendix A Software environment and installation procedure

Component	Version / role
Operating system	Linux (Ubuntu 24.04) under WSL2
OpenFOAM	v2506 (OpenCFD distribution)
OFFBEAT	*master* branch, compiled from source
Python	3.12, dedicated virtual environment
Language model	7 billion parameters, run locally
Embedding model	local **multilingual** model (see §4.6)
Post-processing	VTK reading library
Surrogate	Gaussian process (Matérn kernel)

Remark. The project and the simulation cases must reside on the native Linux file system: the input/output of an OpenFOAM case is severely degraded on a file system mounted from the host.

Appendix B Structure of an OFFBEAT case and exposed parameters

An OFFBEAT case is a file tree:

Element	Contents
`0/`	Initial fields (temperature, displacement, neutron flux)
`constant/`	Material properties, physical models, power history, coolant pressure
`system/`	Run control (duration, time step, writing), numerical schemes and solvers
`rodDict`	Rod geometry by blocks (radii, heights, discretisation), specific to OFFBEAT

Parameters exposed by the agent:

Parameter	Meaning	Unit
`linear_heat_rate`	Linear heat rate	W/m
`end_time`	Simulated duration	s
`delta_t`	Initial time step	s
`write_interval`	Writing frequency	steps
`enrichment`	Enrichment	–
`fuel_outer_radius`	Pellet outer radius	mm
`fuel_height`	Height of the column	mm
`clad_inner_radius`	Cladding inner radius	mm
`clad_outer_radius`	Cladding outer radius	mm
`n_cells_r_fuel`	Radial discretisation	–

Consistency constraint checked before execution: $r_{\text{pellet}} < r_{\text{clad,in}} < r_{\text{clad,out}}$.

Appendix C Error knowledge base

Each entry associates a detection pattern with a diagnosis and a correction. Extract of the empirically validated entries:

Identifier	Diagnosis	Correction
Floating-point exception	Division by zero or non-numeric value: divergence, unrealistic conditions, or a material property out of range	Reduce the time step
Duration beyond the power history	The simulated duration exceeds the last tabulated point of the history	Read the time tables present in the case and bring the duration just below the last point
Restart impossible	Boundary condition not re-readable from a written time step	Clean the time steps and restart cleanly
Inverted geometry	Invalid mesh	None: intervention required
Missing dictionary	Entry absent from a configuration file	None: intervention required

Remark. An entry inherited from the reference project, relating to the Courant number, turned out to be irrelevant to OFFBEAT: the solver contains no advection term, hence no stability condition of that kind. It was kept disabled, together with its justification, as a record.

Appendix D Safety criteria knowledge base

Criterion	Quantity	Limit	Remark
Centreline melting	Temperature, maximum	3113 K	Unirradiated UO_2; the limit decreases with burnup (see below)
PCMI strain	**Plastic** $\theta\theta$ strain, cladding zone	1 %	Usual design criterion; see [7], [8]
Permanent strain	Equivalent creep strain, cladding zone	1 %	Limit taken over by default, **not validated**
Cladding stress	$\theta\theta$ stress, cladding zone	`sigmaY` field	Limit read from the field computed by the solver (fallback 250 MPa)
Gap closure	Gap width, minimum	5 µm	Reversed sense: the danger is a value that is too *small*

Important. These five criteria are marked as **not validated** in the base. They are published design orders of magnitude, not invented values, but their validation with the supervisor remains a prerequisite to any exploitation.

The decrease of the melting limit with burnup. The melting temperature of UO_2 decreases during irradiation: the fission products and the plutonium formed enter into solid solution in the

matrix and lower the melting point, as a solute lowers that of a solvent. The margin to melting therefore narrows over the cycle, at the very moment when the centreline temperature is itself rising.

The note initially carried in the base indicated a slope of -32 K per 10 GWd/tU. The bibliographic check carried out for this report showed that there are in fact **two published correlations**, and that the one adopted by the project was the older of the two (table 3).

Table 3: The two published correlations for the decrease of the melting temperature of UO_2 with burnup. Both start from 3113.15 K for unirradiated UO_2 (Brassfield, 1968).

Correlation	Subroutine	Slope	T_{melt} at 50 GWd/tU
MATPRO (earlier)	`FHYPRP`	-3.2 K/GWd/tU	2953 K
FRAPCON-4.0 / FRAPTRAN-2.0 (adopted)	`PHYPRP`	-0.5 K/GWd/tU	3088 K

The correlation of the current reference codes reads $T_{\text{melt}} = 3113.15 - 5\,Bu/10000$, with Bu in MWd/tU. It **supersedes** the MATPRO one: the revision, by a factor of 6.4, was proposed by Popov *et al.* (2000) on the basis of experimental data from Adamson *et al.* (1985) and Komatsu *et al.*, the steep MATPRO slope having proved too pessimistic [11]. It is therefore the value of -0.5 K/GWd/tU that has been adopted.

Practical significance. At 50 GWd/tU, the discrepancy between the two correlations reaches 135 K on the limit. On the reference case of this report, where the centreline temperature peaks at 1467 K, both give the same verdict with a comfortable margin. The question only becomes discriminating for a rod pushed to high burnup — that is, precisely the domain where safety analysis has the most value.

Appendix E Training dataset of the surrogate

Full design of experiments: 5 values of linear heat rate $\times$ 5 irradiation durations, that is 25 OFF-BEAT simulations, all of them carried through. Total cost: about 30 minutes of computation.

Variable	Domain explored	Role
Linear heat rate	12, 18, 24, 30, 36 kW/m	input
Irradiation duration	7.9×10^6 s to 6.3×10^7 s (3 months to 2 years)	input
Maximum temperature	930 K to 2080 K	output
Maximum creep strain	9.8×10^{-4} to 1.1×10^{-2}	output
Minimum gap width	-13.2 µm to 31.8 µm	output

Choice of the durations. The five values are placed just short of the tabulated points of the template's power history. A duration that reaches or exceeds the last time marker makes the solver fail, for want of a following marker to drive the adaptive time step.

Sign of the outputs. A negative gap width signals an interference, that is, an established pellet-cladding contact. It is precisely because the domain covers this change of sign that the surrogate can learn to date the onset of PCMI — which the first version, limited to durations of a few thousand seconds, did not allow (§4.3).

Range of validity. The surrogate is reliable only within this domain. Any prediction outside it is

extrapolation; this is why the uncertainty of the Gaussian process is systematically displayed next to the predicted value.

Appendix F Catalogue of faults in the self-healing benchmark

Each fault is injected into a *healthy* case generated beforehand, so that the corruption is the only variable. The geometric faults are injected after creation, deliberately to bypass input validation, the purpose of the benchmark being to measure self-healing and not the upstream safeguard.

Category	Injected fault	Correction in base	Cause treatable
Control	Nominal case 25 kW/m	—	—
	Nominal case 18 kW/m	—	—
Numerical	Power 200 times nominal	yes	no
	Power 100 times nominal	yes	no
	Negative power	yes	no
Physical consistency	Pellet wider than the inside of the cladding	no	no
	Negative cladding thickness	no	no
	Zero pellet radius	no	no
Missing file	Main solver dictionary deleted	no	no
	Initial temperature field deleted	no	no
	Control dictionary deleted	no	no
Invalid entry	Non-numeric simulated duration	no	no
	Duration beyond the power history	**yes**	**yes**
	Negative time step	no	no

Reading the last two columns. “Correction in base” indicates that the knowledge base associates a remedy with the expected pattern; “cause treatable” indicates that this remedy can, by its nature, address the root cause. A single fault in the catalogue meets both conditions, and it is the only one that self-healing resolves (§4.4).

Appendix G Worked example: a complete session

Transcript of a real session, captured after the corrections of section 4.5.4. Tool results are abridged; the sequence of calls is complete. The session was conducted in French; the request is translated here.

User request

> *Create a PWR fuel rod case in* `/tmp/demo_agent` *with a linear heat rate of 22 kW/m over about one year, run the simulation, then tell me whether the safety margins are met.*

#	Tool called	Result
1	`input_creator`	Case created, but the agent placed `template_name` *inside* the parameters: the key is reported as ignored
2	`offbeat_executor`	Rejected: "is not a valid OpenFOAM case". The agent had launched the execution *in parallel* with the creation, before the case existed
3	`input_creator`	The agent corrects itself: `template_name` is moved back outside the parameters. Case created
4	`offbeat_executor`	Mesh generated, simulation successful on the first attempt
5	`data_processor`	Axial profile 968.7 K to 1263.5 K; radial profile centre 1220.6 K, periphery 601.6 K; hoop stress −38.81 MPa; PCT 634.5 K; three figures generated

What this session shows. Step 2 is the most instructive: the agent makes a sequencing error — it launches the solver before the case exists — and it is the *deterministic safeguard* of the execution tool, not the language model, that intercepts it. The agent reads the error message, infers the correction and resumes. This is exactly the division of roles the architecture aims at: the model orchestrates, the tools validate.

What it also shows. The request comprised four steps; the agent carried out three. It announced that it was going to launch the safety analysis, then ended its turn without doing so. Long chaining therefore remains imperfect, something the tool-selection benchmark did not measure either: it evaluates only the first call, never the conduct of a sequence.